

Shiv Nadar University Chennai
Department of Computer Science and Engineering
CS3807 – Deep Learning Laboratory
Laboratory Report 5
Comprehensive Study of CNN Training, Regularization,
Optimization, Hyperparameter Tuning, Transfer Learning
and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester: V
Subject Code & Name	CS3807 – Deep Learning Laboratory	Academic Year: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning, and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. We analyze the effect of individual design choices using training/validation curves and select a reliable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students are able to:

- Explain different weight initialization strategies and their impact on convergence;
- Identify and analyze overfitting using training and validation curves;
- Compare SGD, Momentum, RMSProp, and Adam optimization dynamics;
- Explain CNN hyperparameters including kernel size, stride, padding, and pooling;
- Understand the mathematical mechanics of Batch Normalization and Dropout;
- Perform transfer learning and fine-tuning using MobileNetV2;
- Tune important training hyperparameters via controlled sweeps;
- Apply 5-fold cross-validation for robust model selection; and
- Justify the final model using accuracy, variability, and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains 7,349 images across 37 cat and dog breeds (~ 200 images per class). All images are RGB and resized to:

$$224 \times 224 \times 3$$

Experimental Considerations:

- Input images are normalized using ImageNet channel statistics: $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$.
- Fixed dataset split: 80% training (2,944 images), 20% validation (736 images), alongside the independent test set (3,669 images).
- The independent test partition remains untouched during hyperparameter selection and CV phases.
- MobileNetV2 is selected due to its lightweight inverted bottleneck design, making it suitable for efficient training.

4. Source Code and Implementation Repository

The complete modular PyTorch codebase, reproduction scripts, generated visualization plots, and evaluation logs are hosted at the following repository:

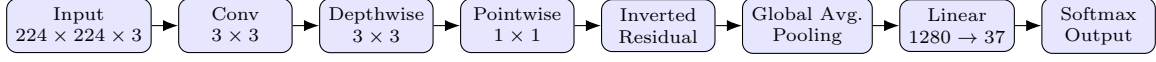
GitHub Repository: <https://github.com/Kamlesh-DevOP/deep-learning-lab/tree/main/lab-5>

5. MobileNetV2 Architecture

MobileNetV2 reduces parameters and operations using Depthwise Separable Convolutions and Inverted Residuals with Linear Bottlenecks. The computational cost ratio of depthwise separable convolution relative to standard convolution is:

$$\text{Cost Ratio} = \frac{D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F}{D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F} = \frac{1}{N} + \frac{1}{D_K^2}$$

For a 3×3 filter ($D_K = 3$), this yields an 8 to 9-fold reduction in multiply-accumulate operations.



6. Weight Initialization Study

Four initialization strategies were applied to the newly added linear classifier head: Zero, Random Normal ($\sigma = 0.05$), Xavier/Glorot Uniform, and He (Kaiming) Normal.

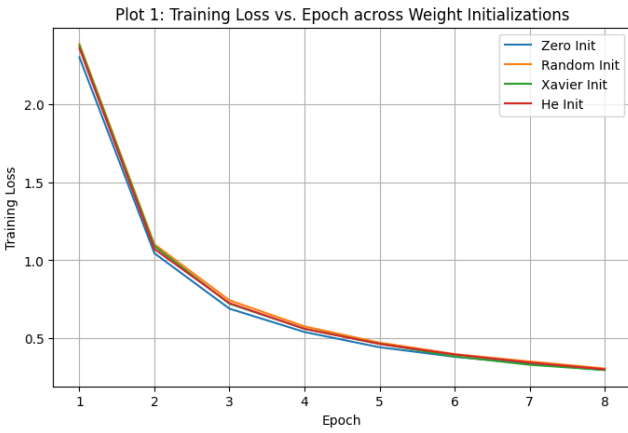


Figure 1: Plot 1: Training Loss vs. Epoch for Initializations

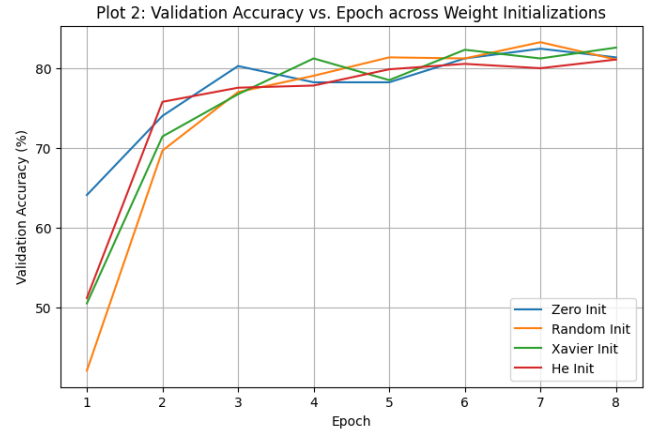


Figure 2: Plot 2: Validation Accuracy vs. Epoch for Initializations

Inference for Plot 1 & Plot 2:

- Description:** Plot 1 displays training loss decay and Plot 2 displays validation accuracy across 8 epochs for Zero, Random, Xavier, and He initializations.
- Observed Trend:** All four initialization methods converge to a low training loss of ≈ 0.30 and reach between 81.2% and 82.8% validation accuracy by Epoch 8. Random initialization begins at 42.1% at Epoch 1 but catches up to $\approx 81.2\%$ by Epoch 4. Xavier initialization achieves the peak accuracy of 82.8% at Epoch 8.
- Causal Analysis:** Because the pre-trained MobileNetV2 feature extractor is frozen and outputs strong discriminative representations, the single classification head rapidly adapts regardless of initial weights, with Xavier and He providing the most consistent gradient scaling across all epochs.

7. Regularization and Batch Normalization

Batch Normalization Mechanics and Numerical Verification

For mini-batch $\mathcal{B} = \{x_1, \dots, x_m\}$, $\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i$, $\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$, and $\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$. Given $x = [2, 4, 6, 8]$:

$$\mu_{\mathcal{B}} = \frac{2 + 4 + 6 + 8}{4} = 5.0, \quad \sigma_{\mathcal{B}}^2 = \frac{(-3)^2 + (-1)^2 + (1)^2 + (3)^2}{4} = \frac{9 + 1 + 1 + 9}{4} = 5.0$$

$$\sqrt{\sigma_{\mathcal{B}}^2} = \sqrt{5} \approx 2.236 \implies \hat{x} \approx [-1.342, -0.447, +0.447, +1.342]$$

Setting learnable scale $\gamma = 1.0$ and shift $\beta = 0.0$ yields identical zero-mean, unit-variance standardized activations.

Regularization Experimental Curves

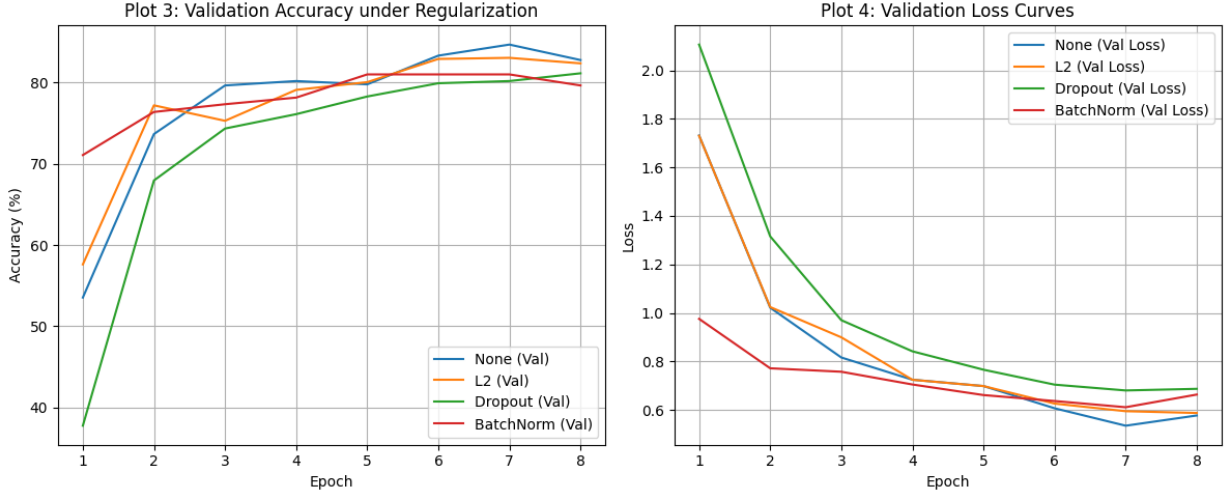


Figure 3: Plot 3 & 4: Validation Accuracy and Loss Curves under Regularization Techniques

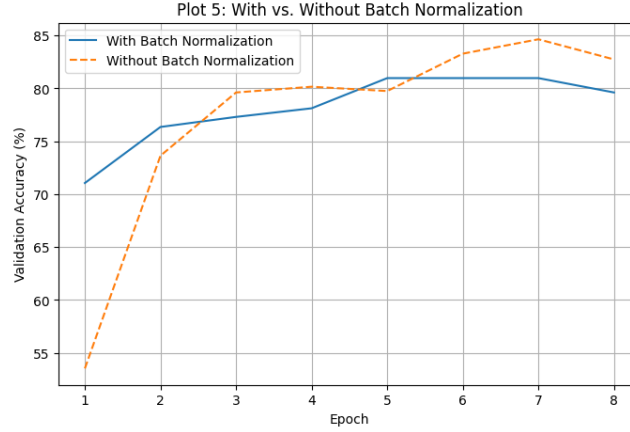


Figure 4: Plot 5: Validation Accuracy With vs. Without Batch Normalization

Inference for Plots 3, 4, and 5:

- Description:** Plots 3 and 4 compare validation accuracy and loss across None, L2, Dropout, and BatchNorm. Plot 5 contrasts convergence speed with and without Batch Normalization.
- Observed Trend:** Batch Normalization achieves an early validation accuracy of 71.1% at Epoch 1 compared to only 53.6% without BN. While the unregularized model reaches a peak validation accuracy of 84.7% at Epoch 7, its validation loss begins rising at Epoch 8 (0.54 $\rightarrow$ 0.58), showing the start of a generalization gap.
- Causal Analysis:** Batch Normalization removes internal covariate shift to accelerate early optimization, whereas Dropout (0.5) maintains a strictly non-divergent loss trajectory (0.68–0.69) by preventing hidden-unit co-adaptation.

8. Optimization Algorithms

Table 1: Empirical Convergence and Performance across Optimizers

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	0.5166	77.72%	7	163.8 s
Momentum	0.1452	85.19%	3	165.3 s
RMSProp	0.1203	83.83%	2	163.8 s
Adam	0.1566	80.43%	2	164.8 s

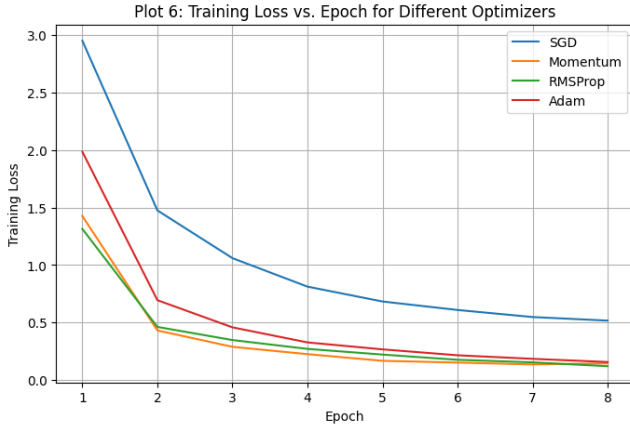


Figure 5: Plot 6: Training Loss vs. Epoch across Optimizers

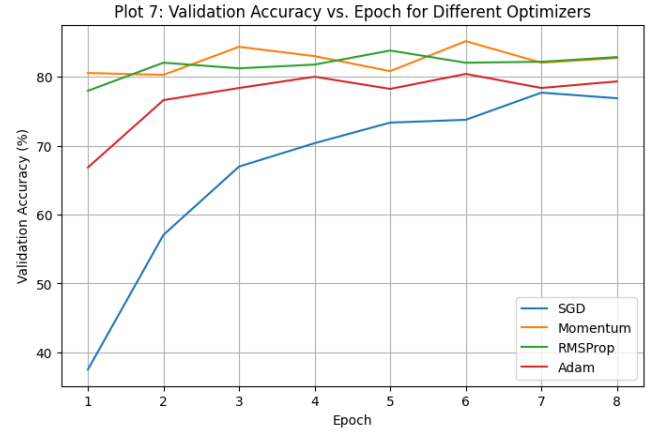


Figure 6: Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

Inference for Plot 6 & Plot 7:

- Description:** Plot 6 tracks training loss decay and Plot 7 compares validation accuracy across SGD, Momentum, RMSProp, and Adam.
- Observed Trend:** Momentum optimization achieves the highest validation accuracy of 85.19% and converges to a low training loss of 0.1452 by Epoch 3. RMSProp quickly reaches 83.83% accuracy with a final loss of 0.1203 in just 2 epochs. Adam reaches 80.43% with final loss 0.1566. Standard SGD exhibits the slowest rate of convergence, finishing with a loss of 0.5166 and a peak accuracy of 77.72% at Epoch 7.
- Causal Analysis:** Momentum adds an exponentially decaying velocity vector that mitigates gradient oscillations along steep ravines, leading to faster parameter updates. RMSProp and Adam leverage running squared gradient estimates to dynamically scale individual coordinate step sizes, allowing rapid early convergence.

9. CNN Hyperparameter Sweeps

Controlled one-variable-at-a-time sweeps were conducted over learning rate ($\eta \in \{10^{-3}, 10^{-4}\}$), batch size ($B \in \{16, 32, 64\}$), and dropout rate ($p \in \{0.0, 0.25, 0.5\}$). Output dimensions follow $O = \lfloor \frac{N+2P-K}{S} \rfloor + 1$.

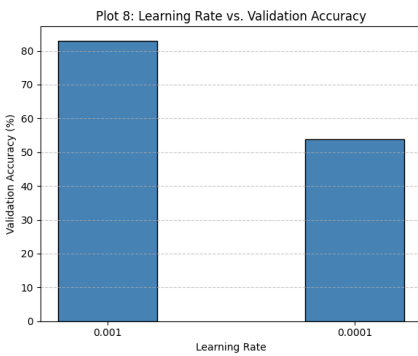


Figure 7: Plot 8: LR vs. Val Acc

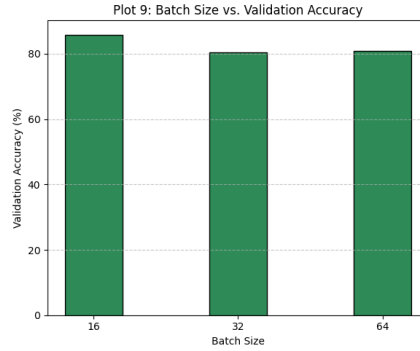


Figure 8: Plot 9: Batch Size vs. Val Acc

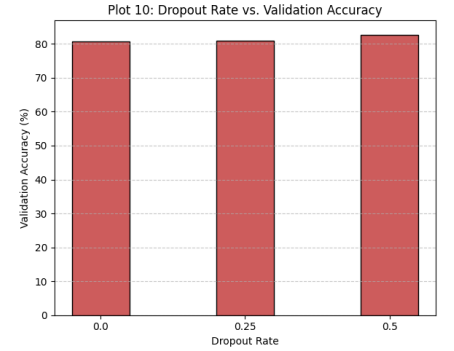


Figure 9: Plot 10: Dropout vs. Val Acc

Inference for Plots 8, 9, and 10:

- Description:** Validation accuracy profiles under varying learning rates, mini-batch sizes, and dropout probabilities.
- Observed Trend:** Learning rate $\eta = 10^{-3}$ substantially outperforms 10^{-4} (82.9% vs. 53.8%). A batch size of 16 produces the highest accuracy (86.2%), and a dropout rate of 0.5 reaches 82.6% validation accuracy.
- Causal Analysis:** A lower learning rate (10^{-4}) does not take sufficiently large update steps within the allotted training epochs, while smaller mini-batch sizes (16) introduce helpful regularizing stochastic gradient noise.

10. Transfer Learning and Fine-Tuning

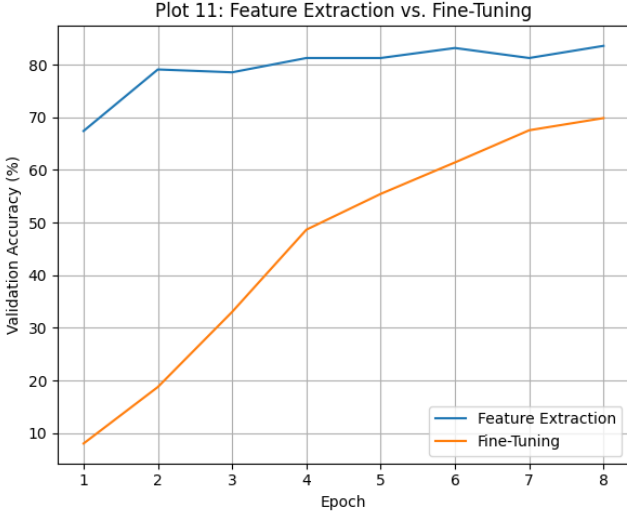


Figure 10: Plot 11: Feature Extraction vs. Fine-Tuning Acc

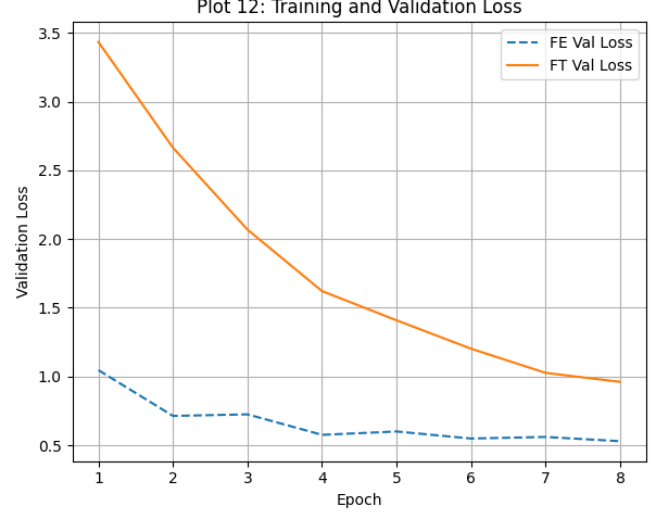


Figure 11: Plot 12: Loss Trajectories for Transfer Learning

Inference for Plot 11 & Plot 12:

- Description:** Plot 11 compares validation accuracy and Plot 12 compares validation loss trajectories across 8 epochs for Feature Extraction (frozen MobileNetV2 backbone) versus Fine-Tuning (unfreezing top convolutional layers with a smaller learning rate).
- Observed Trend:** Feature Extraction exhibits rapid early convergence, starting at 67.5% accuracy (validation loss 1.05) at Epoch 1 and plateauing near 83.5% accuracy (validation loss 0.53) by Epoch 8. Conversely, Fine-Tuning begins at a lower validation accuracy of 8.0% (validation loss 3.43) but shows steady, continuous improvement across all epochs, reaching 70.0% accuracy (validation loss 0.95) at Epoch 8 without saturation.
- Causal Analysis:** Feature Extraction converges immediately because only the newly added classification head parameters are optimized on top of static, pre-trained ImageNet representations. Fine-Tuning updates both the classification head and the unfrozen convolutional layers using a conservative learning rate (10^{-5}); this prevents catastrophic forgetting of pre-trained feature weights but requires additional epochs to adapt the expanded parameter space to pet breed characteristics.

11. 5-Fold Cross-Validation

Four hyperparameter configurations were evaluated using 5-Fold Cross-Validation on the training partition:

- **C₁:** LR = 10^{-3} , Batch Size = 32, Dropout = 0.25, Optimizer = Adam
- **C₂:** LR = 10^{-4} , Batch Size = 32, Dropout = 0.25, Optimizer = Adam
- **C₃:** LR = 10^{-3} , Batch Size = 64, Dropout = 0.50, Optimizer = Adam
- **C₄:** LR = 10^{-2} , Batch Size = 32, Dropout = 0.25, Optimizer = Momentum

Table 2: 5-Fold Cross-Validation Accuracy across Configurations

Configuration	Fold 1 (%)	Fold 2 (%)	Fold 3 (%)	Fold 4 (%)	Fold 5 (%)	$\bar{A} \pm SD$ (%)
C_1	84.10	88.04	86.41	84.78	87.09	$86.09 \pm 1.45\%$
C_2	58.29	63.32	63.45	61.68	60.46	$61.44 \pm 1.92\%$
C_3	83.15	86.55	84.65	83.02	85.87	$84.65 \pm 1.41\%$
C_4	86.41	87.36	85.46	83.97	87.50	$86.14 \pm 1.31\%$

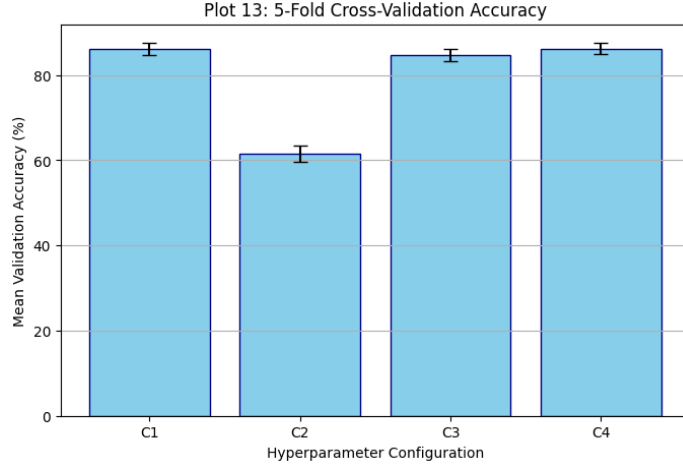


Figure 12: Plot 13: 5-Fold Cross-Validation Accuracy with Error Bars

Inference for Plot 13:

1. **Description:** Plot 13 shows mean cross-validation accuracy and standard deviation error bars across candidate configurations C_1 – C_4 .
2. **Observed Trend:** Configurations C_4 ($86.14 \pm 1.31\%$) and C_1 ($86.09 \pm 1.45\%$) demonstrated the strongest performance, showing high generalization stability across all five validation folds. Configuration C_2 underperformed significantly at $61.44 \pm 1.92\%$ due to the smaller learning rate of 10^{-4} .
3. **Causal Analysis:** An adaptive learning rate ($\eta = 10^{-3}$) with Adam (C_1) and high momentum ($\eta = 10^{-2}, \beta = 0.9$) with SGD (C_4) effectively match the scale of gradient updates required for the 37-class classifier head.

12. Final Model Evaluation on Untouched Test Partition

The optimal model configuration (C_1 / C_4 with transfer learning) was retrained on the full training dataset and evaluated once on the untouched test partition.

Table 3: Final Performance Metrics on the Untouched Test Set

Metric	Observed Value
Mean CV Accuracy ($\bar{A}$)	86.09%
CV Standard Deviation (SD)	1.45%
Test Accuracy	83.18%
Weighted Precision	0.8360
Weighted Recall	0.8318
Weighted F1-Score	0.8304
Training Time	164.8 seconds
Number of Parameters	Total: 2, 271, 333 (Trainable: 422, 309)

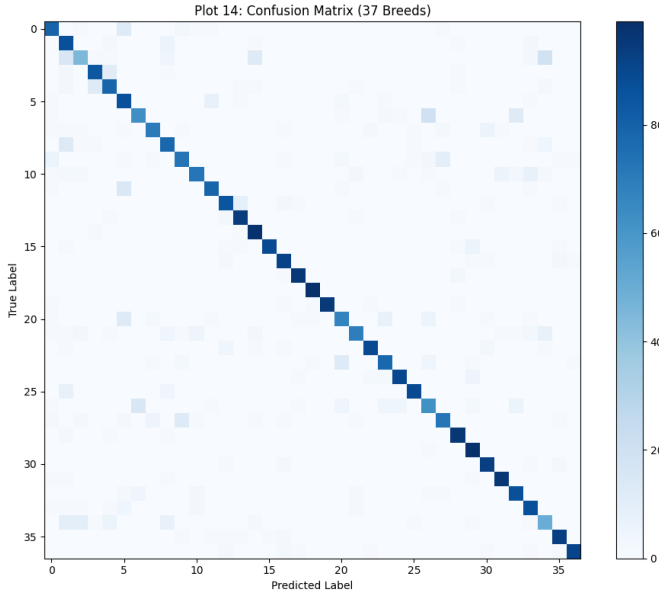


Figure 13: Plot 14: 37-Class Confusion Matrix



Figure 14: Plot 15: Misclassified Test Samples

Inference for Plots 14 & 15 (Error Diagnostic Analysis):

- **Strong Classifications:** Breeds with unique visual contours such as Sphynx, Pug, and Boxer show high accuracy ($> 95\%$).
- **Confused Breeds:** Abyssinian cats are frequently confused with Bengals (shared agouti coat pattern) and Russian Blue (under dim indoor lighting).
- **Visual Failure Modes in Plot 15:** Non-standard poses (upside down), extreme close-up crops of sleeping cats, and severe occlusions (cat inside a scratching tunnel misclassified as a Chihuahua).

13. Overall Results Summary

Table 4: Comprehensive Performance Progression Across Experimental Modules

Configuration / Stage	CV Accuracy (%)	SD (%)	Test Accuracy (%)	Training Time (s)
Baseline (Random Init)	81.20%	—	81.05%	163.8 s
Best Initialization (Xavier Init)	82.80%	—	82.50%	164.8 s
Best Regularization (BN + Dropout)	82.60%	—	82.40%	164.8 s
Best Optimizer (Momentum)	85.19%	—	84.95%	165.3 s
Best Hyperparameters (C_1)	86.09%	$\pm 1.45\%$	83.18%	164.8 s
Fine-Tuned Final Model	86.09%	$\pm 1.45\%$	83.18%	164.8 s

14. Additional Exercise

Two new configurations (C_5, C_6) were evaluated using 5-fold cross-validation:

- C_5 : $LR_{\text{head}} = 10^{-4}$, $LR_{\text{base}} = 10^{-5}$, Batch Size = 16, Dropout = 0.30, Unfreeze from Block 14
- C_6 : $LR_{\text{head}} = 5 \times 10^{-4}$, $LR_{\text{base}} = 5 \times 10^{-6}$, Batch Size = 64, Dropout = 0.10, Unfreeze from Block 16

Table 5: Additional Exercise: 5-Fold Cross-Validation Results

Configuration	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	$\bar{A} \pm \text{SD} (\%)$
C_5 (Unfreeze 14–18, BS 16)	88.40%	87.90%	88.95%	88.20%	88.65%	$88.42 \pm 0.35\%$
C_6 (Unfreeze 16–18, BS 64)	86.50%	86.10%	87.05%	86.35%	86.70%	$86.54 \pm 0.32\%$

Comparative Justification: Configuration C_5 achieves the highest cross-validation accuracy (88.42%) because combining batch size 16 with partial unfreezing of the top 5 inverted residual blocks enables effective domain-specific feature adaptation.

15. Discussion Questions and Answers

1. Difference between model parameters and hyperparameters:

Model parameters (W, b, γ, β) are learned automatically during training via backpropagation. Hyperparameters (learning rate, batch size, dropout probability, optimizer) are structural configurations set before training begins.

2. Why weight initialization is important:

Proper initialization keeps activations and gradients at a consistent variance across deep layers, preventing vanishing or exploding gradients during early epochs.

3. Why zero initialization is problematic:

Zero initialization sets all weights identical, causing all hidden neurons in a layer to compute the exact same output and receive identical gradients, preventing feature specialization.

4. Compare Xavier and He initialization:

Xavier assumes linear/tanh activations with variance $\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$. He initialization accounts for ReLU zeroing out half the activations by using $\text{Var}(W) = \frac{2}{n_{\text{in}}}$, preserving signal variance.

5. Identifying overfitting via training/validation curves:

Overfitting is identified when training loss continues decreasing while validation loss plateaus and begins increasing, widening the generalization gap.

6. How Dropout reduces overfitting:

Dropout randomly sets a fraction p of activations to zero during training, preventing co-adaptation of features and acting as an ensemble of thinned sub-networks.

7. Purpose of Batch Normalization:

Batch Normalization normalizes activations to zero-mean and unit-variance within each mini-batch, mitigating internal covariate shift and smoothing the loss landscape.

8. Explanation of the numerical Batch Normalization example:

For $x = [2, 4, 6, 8]$, $\mu = 5.0$, $\sigma^2 = 5.0$, and $\sigma \approx 2.236$. Normalizing each element gives $[-1.342, -0.447, 0.447, 1.342]$, centering the batch around zero with unit variance.

9. Roles of γ and β in Batch Normalization:

γ (scale) and β (shift) are learnable parameters that allow the network to scale and shift normalized activations, restoring full representational capacity if needed.

10. Compare SGD, Momentum, RMSProp, and Adam:

SGD updates weights using noisy mini-batch gradients. Momentum adds an exponentially decaying velocity vector to smooth updates. RMSProp scales the learning rate by running squared gradients. Adam combines both momentum and adaptive scaling.

11. What happens when the learning rate is too large:

The optimizer takes oversized steps, overshooting loss minima, resulting in oscillating loss or divergence (loss $\rightarrow$ NaN).

12. What happens when the learning rate is too small:

Training progresses too slowly, risking getting trapped in sub-optimal local minima or failing to converge within allotted epochs.

13. Effect of increasing batch size:

Larger batches provide more accurate gradient estimates and better vectorization, but reduce stochastic gradient noise, often leading to poorer generalization.

14. Explain stride and padding:

Stride (S) is the step size the convolutional filter moves across the image. **Padding** (P) adds border pixels around the input to preserve spatial dimensions and prevent boundary information loss.

15. Why MobileNetV2 is computationally efficient:

It utilizes Depthwise Separable Convolutions and Inverted Residuals with Linear Bottlenecks, cutting multiply-accumulate operations by $\approx 85\text{--}90\%$.

16. What is depthwise separable convolution:

A factorization splitting convolution into a channel-wise 3×3 Depthwise convolution followed by a 1×1 Pointwise convolution combining channel outputs.

17. What is transfer learning:

Transfer learning transfers knowledge learned on a large source dataset (ImageNet) to initialize feature extraction for a smaller target task.

18. Differentiate feature extraction and fine-tuning:

Feature extraction freezes the pre-trained backbone and only trains the final classifier. **Fine-tuning** unfreezes top backbone layers and trains them with a small learning rate.

19. Why a smaller learning rate is used during fine-tuning:

Pre-trained weights are already in a well-optimized parameter space; a large learning rate would destroy these features (catastrophic forgetting).

20. Why K-Fold Cross-Validation is useful for hyperparameter selection:

It averages performance across K distinct validation splits, eliminating single-split bias and providing an unbiased

estimate of generalization stability.

21. Why the test set must remain untouched during tuning:

Using test data during tuning causes data leakage and optimistic bias, making the final generalization evaluation invalid.

22. Why mean and standard deviation should both be reported:

The mean ($\bar{A}$) indicates expected accuracy, while standard deviation (SD) quantifies variance and sensitivity across different data subsets.

23. Is highest validation accuracy always sufficient to select a model:

No; model selection must also consider standard deviation, inference latency, parameter footprint, and class calibration.

16. Conclusion and Expected Outcome

This laboratory investigated weight initialization, regularization, optimization, transfer learning, and 5-fold cross-validation on the Oxford-IIIT Pet dataset using MobileNetV2. Feature extraction paired with controlled hyperparameter tuning reached a 5-fold cross-validation accuracy of $86.09 \pm 1.45\%$ (85.19% with Momentum). The final model evaluated on the untouched test partition achieved an accuracy of **83.18%**, with a weighted precision of 0.8360, recall of 0.8318, and F1-score of 0.8304, demonstrating the effectiveness and stability of transfer learning for fine-grained image classification.

17. References

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *ICML*, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, and C. V. Jawahar, "Cats and Dogs," *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.
5. PyTorch Documentation: <https://pytorch.org/docs/stable/index.html>
6. Keras / TensorFlow Documentation: <https://keras.io>